Asking loan-level RMBS data questions in plain English

Utkarsh Malhotra · November 2026

The setting

The question: monthly trustee data for about 300 Australian RMBS deals sits in Oracle. Each deal holds about 2,000 loans and goes back 5 years, so the database has about 36 million loan-month records. Each record carries arrears status, prepayments, LVR, interest rate, balance and a suburb that links to local house prices. How do you let people ask questions of that data in plain English, and trust the answers?

The size is not the hard part. A well partitioned table of 36 million rows answers an aggregate query in seconds, and the prototype below backs that up. The hard part is meaning. "What are arrears on the 2021 vintage in Western Sydney?" could mean 30 or 90 days past due, by loan count or by balance, at the latest month or averaged over a year, with hardship loans in or out. A model writing free SQL against raw trustee tables picks 1 of those readings without telling you, and returns a confident number. On a credit desk, a confident wrong number is worse than no number.

The short answer

Keep the language model away from the arithmetic.

The model's only job is translation. It turns the question into a small, structured query spec: which metric, grouped by what, filtered how, over which months. A compiler turns that spec into SQL over metric definitions written once in code. The database does the maths. The model then writes a short answer from the finished table, and a guard checks that every number in that answer appears in the table. If the question is ambiguous, the system asks. If the data can't answer it, the system says so.

Every answer ships with its receipt: the spec, the Oracle SQL and the table. An analyst can check any answer in about 10 seconds.

The design at production scale

1. Fix the data before any model touches it. Trustee reports from different trustees don't mean the same thing.

- One reports arrears in days and another in missed payments.
- Hardship may sit inside or outside arrears.
- A clean-up call can arrive coded as an ordinary repayment.
- Corrected months get resubmitted on top of the originals.

So the first layer is 1 canonical loan-month table, with a stable loan key and every exit reason tagged separately: full prepayment, repurchase, default and call. Indexed LVR is computed once, as current balance over original valuation scaled by the suburb house price index, and stored as a column. Skip this layer, and the system will one day count a clean-up call as a wave of prepayments and report a CPR spike that never happened.

2. A semantic layer of governed metrics. Each metric is defined once, in code. Examples:

- 90+ arrears by balance, hardship excluded;
- CPR from voluntary prepayments only, compounded over the period;
- balance-weighted indexed LVR.

The dimensions (deal, originator, vintage, state, suburb, LVR band, rate type, interest only) are listed with their allowed values. The model can only pick from these lists, so it can't invent a metric.

3. A spec, not SQL. The model fills a constrained schema: 1 to 3 metrics, up to 2 groupings, filters, a time window and an optional ranking. Filling a schema is far more reliable than writing correct window functions over a loan panel that thins out as loans repay. The spec is validated before anything runs.

4. Resolve names, then compile.

- Names people actually type, like "Harbour 21-1", "blacktown" or "HARB21-1", are matched to stored values. Exact matches and known aliases resolve silently. Close misspellings resolve with a score. A name that fits several values comes back as a question.
- The compiler then writes 1 SQL statement for DuckDB and renders the same statement for Oracle, avoiding constructs Oracle can't run.

5. Narrate from the result only, and check it. The answer text is written from the finished table. A guard pulls every number out of the text and rejects any that isn't in the table. When the guard trips, the user gets a plain table summary instead.

6. Measure it. A fixed set of real questions, each with a hand-checked answer, runs on every change to the prompts, the model or a metric definition. The set includes ambiguous questions, to check the system asks rather than guesses. Without this, you can say the system sounds accurate. You can't say it is.

At 36 million rows, most questions land on deal-month or month-level aggregates, so a small precomputed table answers them, and loan-level detail stays available for drill-downs. In production I'd put it behind a read-only replica, with query timeouts and per-deal permissions for anything held under NDA.

What the prototype shows

The prototype runs the whole design end to end on synthetic data built to carry the same traps: 50 deals, 21,945 loans and 773,336 loan-months from 2021-10 to 2026-09, delivered in 2 different trustee formats. Rates follow the RBA path, house prices follow suburb indices, and 90+ arrears rise from 0.18% to a peak of 1.54% in 2024-08, then ease to 0.47%.

The cleaning is proved, not assumed. The raw trustee files hold 774,057 rows, including 721 resubmitted duplicates. One format codes its 7 clean-up calls as ordinary repayments. After cleaning, the canonical table matches the simulated truth on every one of its 773,336 loan-months, with 0 mismatches.

The compiler is checked against hand-written answers. 49 of 49 hand-written SQL queries match the compiler's output, and the worst difference is 1.6e-15. Every compiled query renders to Oracle without a construct Oracle can't run.

Size isn't the problem. Copied 36 times to 27,840,096 loan-months, each of the 34 answerable eval questions runs in under 0.75 seconds on 1 machine with DuckDB's default threads. The slowest, on a single thread, takes 3.34 seconds. These are DuckDB timings; Oracle timings depend on partitioning and indexes, and I didn't measure them.

Against plain text-to-SQL. The eval compares the governed system with a model writing SQL directly, on the same 40 questions with the same table definitions and metric formulas. It hasn't been run in this copy, because a run needs an Anthropic API key and none is stored here. It takes 2 commands and about 114 API calls, and every document, this one included, regenerates with the results:

```
$env:ANTHROPIC_API_KEY = "<your key>"
python scripts/run_eval.py --mode record
python scripts/build_writeup.py
```

The traps the eval is built around

The 8 trap questions target mistakes that look fine in SQL and are wrong in credit terms. On this data:

- counting clean-up call exits as prepayments turns CST 2019-1's 2025 CPR from 21.6% into 100.0%;
- counting exit rows as live loans adds 597 phantom loans to the pool in November 2025;
- a simple average instead of balance weighting moves the pool's LVR from 51.96% to 50.92%.

A fourth candidate, averaging monthly CPR instead of compounding it, was dropped: at the pool's prepayment rate of 18.1%, the 2 methods differ by 1.2e-05, so it couldn't tell a careful system from a careless one.

What I'd build next, and the limits

Next comes a set of analysis tools beyond aggregation:

- roll rate matrices;

- prepayment curves by seasoning;
- cohort loss curves;
- arrears sensitivity to house price falls.

The last needs care. From 2022 to 2024, rate rises pushed arrears up and house prices down in the same suburbs at the same time, so a naive regression would credit house prices with a rate effect. That tool should control for payment shock and say plainly what it can't separate.

The eval set should also grow from real usage: log every question, and add every failure.

The limits:

- the data is synthetic;
- the eval is 40 questions run once at temperature 0, so each category result carries a wide interval, and repeat-run stability isn't measured;
- Oracle SQL is rendered and syntax-checked, but not executed against Oracle.

All figures are generated from CSV files committed in the repo. The data is synthetic.